EvoPolicy-SDD: Verificação Orientada por Especificações para a Evolução Segura de Regras de Negócio em Sistemas com Agentes de IA

Lucas Matheus Silva[1]

[1]Instituto Federal da Paraíba (IFPB)
Laboratório de Inovação em Software (LIS)

Abstract. *LLM-based agents increasingly support business decisions, such as approving a refund, by combining rules written in code with policy documents retrieved as justification. When a policy changes, these two artifacts are maintained separately and may stop saying the same thing: the agent keeps running without errors, but its decisions contradict their own justification. We call this coevolution drift. This research proposal presents EvoPolicy-SDD, a specification-driven approach that binds requirements, rules and documents in a versioned specification and blocks decisions when they diverge. A deterministic prototype (35 automated tests) and an exploratory mutation study show that conventional versioning detects 2 of 12 injected drifts, while the proposed structural checks detect 10 of 12, with one false alarm in 6 benign changes. The two missed drifts change meaning without changing structure, which motivates the core of the planned study: LLM-assisted semantic verification between rule and text, evaluated with real agents and systematic mutations.*

Resumo. *Agentes baseados em LLMs apoiam cada vez mais decisões de negócio, como aprovar um reembolso, combinando regras escritas em código com documentos de política recuperados como justificativa. Quando a política muda, esses dois artefatos são mantidos separadamente e podem deixar de dizer a mesma coisa: o agente continua funcionando sem erros, mas suas decisões passam a contradizer a própria justificativa. Neste trabalho, esse fenômeno é chamado de deriva de coevolução. Este projeto de pesquisa apresenta o EvoPolicy-SDD, uma abordagem orientada por especificações que amarra requisitos, regras e documentos numa especificação versionada e bloqueia decisões quando eles divergem. Um protótipo determinístico (35 testes automatizados) e um estudo exploratório de mutações mostram que o versionamento convencional detecta 2 de 12 derivas injetadas, enquanto as verificações estruturais propostas detectam 10 de 12, com um alarme falso em 6 mudanças benignas. As duas derivas não detectadas mudam o sentido sem mudar a estrutura, o que motiva o núcleo do estudo planejado: a verificação semântica entre regra e texto assistida por LLM, avaliada com agentes reais e mutações sistemáticas.*

1. Introdução

Empresas têm usado agentes baseados em Modelos de Linguagem de Grande Escala (LLMs) para apoiar decisões operacionais, como aprovar um reembolso, liberar um crédito ou encaminhar um atendimento [Yao et al. 2023, Wang et al. 2024]. Para que essas decisões sigam as políticas da organização, os agentes costumam combinar duas fontes: *regras de negócio escritas em código*, que garantem decisões previsíveis,

e *documentos de política em linguagem natural*, recuperados por busca semântica (*Retrieval-Augmented Generation*, RAG) e apresentados como justificativa da decisão [Lewis et al. 2020, Gao et al. 2023]. Esse desenho híbrido também atende a exigências de auditoria e de governança de IA [National Institute of Standards and Technology 2023, União Europeia 2024, Raji et al. 2020].

O problema surge quando a política muda. A regra no código e o texto do documento descrevem a mesma política, mas são mantidos em lugares diferentes, por pessoas diferentes e em momentos diferentes. Se apenas um deles for atualizado, o agente continua funcionando sem nenhum erro, mas passa a tomar decisões que contradizem a própria justificativa. Neste projeto, essa situação é chamada de **deriva de coevolução**: regra executável e documento de evidência deixam de dizer a mesma coisa após uma mudança de política. A Seção 2 ilustra o problema com um exemplo.

Esse tipo de divergência entre artefatos acoplados é conhecido em Engenharia de Software, por exemplo entre código e comentários [Tan et al. 2007, Wen et al. 2019] ou entre código e documentação [Aghajani et al. 2019]. Em agentes de IA, porém, a consequência é mais grave, porque o texto divergente é entregue ao usuário como fundamento da decisão. Trabalhos recentes já transformam políticas em código de guarda para agentes [Zwerdling et al. 2025, Datla et al. 2025], mas tratam uma única versão da política; os próprios autores de [Zwerdling et al. 2025] apontam a gestão do ciclo de vida, quando política ou código mudam, como questão em aberto.

Este projeto propõe o *EvoPolicy-SDD*, uma abordagem orientada por especificações (*Spec-Driven Development*, SDD) em que requisitos, regras e documentos são amarrados numa especificação versionada, e o agente só publica decisões quando esses artefatos estão consistentes entre si. A pesquisa busca responder: *como manter auditáveis as decisões de agentes híbridos quando as políticas de negócio evoluem, e até onde a verificação estrutural e a verificação semântica conseguem detectar a deriva?*

As contribuições pretendidas são: (i) a caracterização da deriva de coevolução em agentes híbridos, com uma taxonomia de tipos de deriva; (ii) um metamodelo e um validador com níveis cumulativos de verificação, que permitem atribuir a detecção a cada mecanismo; (iii) uma verificação semântica entre regra e texto assistida por LLM, voltada às derivas que a verificação estrutural não alcança; e (iv) um estudo empírico, baseado em mutações, sobre ganhos, custos e limites dessas verificações. Um protótipo determinístico já implementa (ii) e uma versão inicial do estudo (iv), cujos resultados estão na Seção 6.

O texto está organizado da seguinte forma: a Seção 2 descreve o problema; a Seção 3 apresenta a fundamentação; a Seção 4 discute trabalhos relacionados e a lacuna; a Seção 5 descreve a abordagem; a Seção 6 reporta a avaliação exploratória; a Seção 7 detalha o plano de pesquisa; a Seção 8 discute ameaças à validade; e a Seção 9 conclui.

2. O Problema: Deriva de Coevolução

Exemplo motivador. Considere um agente que apoia atendentes de uma plataforma fictícia de pedidos em decisões de reembolso. Na versão 1 da política, o cliente que desiste do pedido depois do despacho não tem direito a reembolso. A regra em código (`desistência após despacho` $\rightarrow$ NEGAR) e o documento da política dizem a mesma coisa, e o agente nega o pedido citando o documento correto.

Na versão 2, a empresa cria uma exceção: clientes *premium* podem cancelar até 1 hora após o despacho. A equipe de conteúdo atualiza o documento, mas a regra em

código não é alterada. A Figura 1 mostra o que acontece: para um cliente *premium* que cancela 40 minutos após o despacho, o agente aplica a regra antiga e **nega** o reembolso, mas cita como justificativa o documento novo, que diz que o reembolso é **permitido**. Nenhum teste falha, nenhum erro é registrado, e a decisão chega ao atendente com uma justificativa que a contradiz.

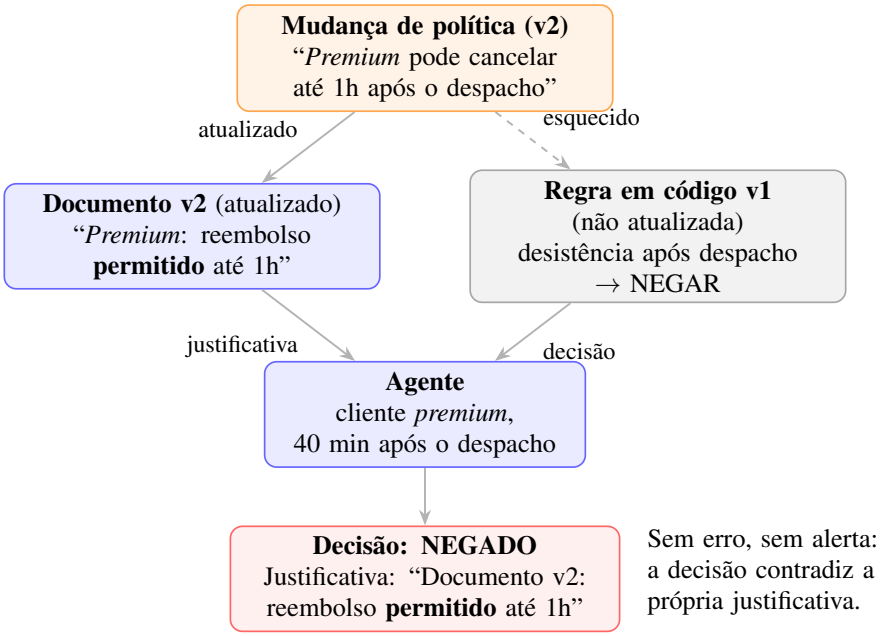


Figura 1. Exemplo de deriva de coevolução: a política muda, o documento é atualizado e a regra em código não.

Tipos de deriva. O exemplo é um de vários casos possíveis. A Tabela 1 resume os tipos considerados neste projeto, que orientam o estudo de mutações da Seção 6.

Tabela 1. Tipos de deriva de coevolução considerados.

Tipo	O que acontece	Efeito na decisão
Regra sem documento	a regra muda (limiar, precedência, decisão) e o texto não	decisão nova com justificativa antiga
Documento sem regra	o texto muda e a regra não (caso da Figura 1)	decisão antiga com justificativa nova
Atualização parcial	parte dos documentos fica na versão anterior	justificativas de versões misturadas
Requisito órfão	um requisito é criado ou removido sem regra correspondente	comportamento sem respaldo no requisito
Deriva semântica	o sentido muda e as marcas de controle são atualizadas sem revisão real	passa por qualquer checagem estrutural

Por que o versionamento convencional não basta. Numerar versões de documentos e de código, como fazem as equipes em geral, detecta apenas o caso em que as versões declaradas diferem. Nos demais, as versões podem coincidir enquanto o conteúdo diverge. Testes de regressão que verificam apenas a decisão final também não ajudam,

porque não olham a justificativa. O problema exige verificar o *vínculo* entre requisito, regra e texto a cada mudança, e não cada artefato isoladamente.

3. Fundamentação

Agentes baseados em LLM e RAG. RAG combina um recuperador e um gerador para que a resposta seja condicionada a documentos externos [Lewis et al. 2020, Gao et al. 2023]. Sistemas RAG apresentam pontos de falha recorrentes de engenharia, como conteúdo ausente e recuperação de trechos inadequados [Barnett et al. 2024], e LLMs podem produzir justificativas plausíveis sem sustentação nas fontes [Ji et al. 2023, Liu et al. 2024].

Especificações como fonte de verdade. Projeto por contrato [Meyer 1992] e engenharia dirigida por modelos [Schmidt 2006] defendem especificações explícitas e verificáveis como fonte do comportamento do software. O SDD retomou esse princípio para guiar agentes de codificação a partir de especificações versionadas [GitHub 2025]. Aqui, o princípio é aplicado a outro alvo: a especificação governa a consistência entre regra executável e texto usado como evidência.

Rastreabilidade e evolução. A rastreabilidade de requisitos liga cada requisito aos artefatos que o implementam [Gotel and Finkelstein 1994, Ramesh and Jarke 2001], e mantê-la sempre atualizada continua sendo um desafio em aberto [Cleland-Huang et al. 2014]. Artefatos acoplados tendem a divergir quando evoluem em ritmos diferentes [Lehman 1980, Zaidman et al. 2011].

Teste de mutação. O teste de mutação avalia a capacidade de detecção de um conjunto de verificações injetando defeitos artificiais [DeMillo et al. 1978, Jia and Harman 2011, Papadakis et al. 2019]; a definição do que conta como defeito exige um oráculo [Barr et al. 2015]. Este projeto adapta a ideia: os operadores de mutação injetam derivas no par especificação–documentos e medem se o validador as detecta.

4. Trabalhos Relacionados

Políticas transformadas em guardas para agentes. [Zwerdling et al. 2025] compilam documentos de política em código de guarda que verifica cada chamada de ferramenta de um agente antes da execução, com testes derivados de trechos do texto da política. [Datla et al. 2025] usam LLMs para traduzir políticas em regras executáveis numa linguagem específica de domínio e comparam o resultado com regras escritas por especialistas. Ambos verificam a correspondência entre texto e regra no momento da construção, para uma única versão da política. *Frameworks* como o NeMo Guardrails [Rebedea et al. 2023] e motores de política como o Open Policy Agent [Open Policy Agent sd] tornam as regras declarativas e versionáveis, mas não as relacionam ao texto apresentado como justificativa.

Políticas e conhecimento que mudam. O PolicyBank [Choi et al. 2026] permite que o agente refine a sua *interpretação* de uma política ambígua a partir de *feedback*, mas a política em si não muda. O *benchmark* HoH [Ouyang et al. 2025] mostra que informação desatualizada na base de recuperação reduz a acurácia de sistemas RAG, mas considera apenas documentos, sem regras executáveis. Na área de processos de negócio, [Hashmi et al. 2018] e [Rinderle et al. 2004] tratam conformidade e mudanças de *workflows*, sem agentes que usam texto como evidência.

Inconsistência entre código e documentação. DocPrism [Xu et al. 2025] e METAMON [Lee et al. 2025] usam LLMs para detectar divergências entre código e

documentação de software. São a base mais próxima para a verificação semântica proposta aqui, mas tratam funções e APIs, não regras de negócio que evoluem com uma política.

Lacuna. A Tabela 2 compara os trabalhos. Nenhum deles trata, ao mesmo tempo, regras executáveis, texto de política usado como evidência e a evolução de ambos ao longo de versões. Essa combinação é o foco do EvoPolicy-SDD. A revisão sistemática prevista na Seção 7 [Kitchenham and Charters 2007] confirmará ou refinará essa lacuna.

Tabela 2. Comparação com trabalhos relacionados (✓: trata; ∼: parcialmente; –: não trata).

Trabalho	Regras executáveis	Texto como evidência	Consistência regra–texto	Evolução de versões
[Zwerdling et al. 2025]	✓	∼	∼	–
[Datla et al. 2025]	✓	–	∼	–
[Choi et al. 2026]	–	✓	–	∼
[Ouyang et al. 2025]	–	✓	–	✓
[Xu et al. 2025, Lee et al. 2025]	–	–	✓	–
EvoPolicy-SDD	✓	✓	✓	✓

5. Abordagem Proposta

Esta seção descreve o metamodelo, o fluxo de verificação e decisão, e as categorias de verificação do EvoPolicy-SDD, tal como implementados no protótipo v0.1.1.

Metamodelo. Cada versão da especificação (`specs/vN/specification.yaml`) declara: o esquema de fatos aceitos (tipos `bool`, `str` e `number`); requisitos com identificador estável e critério de aceitação; regras com prioridade, condição, decisão, justificativa, requisitos atendidos e documento associado; uma decisão padrão; e o *digest* do texto aprovado de cada documento. A base documental (`knowledge/vN/policies.json`) traz versão explícita e, para cada regra vinculada, uma marca de integridade (*fingerprint*) SHA-256 [National Institute of Standards and Technology 2015] calculada sobre os atributos estruturados da regra, incluindo a justificativa.

Fluxo de verificação e decisão. A Figura 2 apresenta o fluxo principal, organizado em seis etapas.

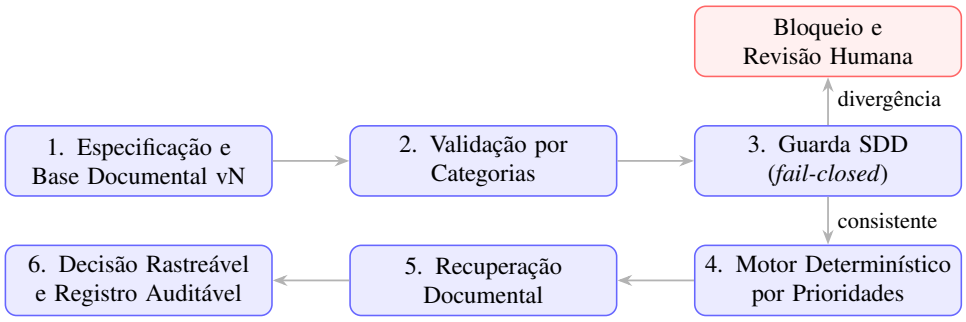


Figura 2. Fluxo de verificação e decisão do EvoPolicy-SDD.

Na *Etapa 1*, a especificação e a base documental de uma versão são carregadas. Na *Etapa 2*, o validador verifica a consistência entre ambas e devolve os problemas encontrados, cada um associado a uma categoria. Na *Etapa 3*, a guarda SDD bloqueia a publicação

se houver qualquer divergência e encaminha os casos para revisão humana (política *fail-closed*). Na *Etapa 4*, com os artefatos consistentes, o motor aplica as regras em ordem de prioridade sobre fatos estruturados, usando a decisão padrão da especificação quando nenhuma regra se aplica. Na *Etapa 5*, o documento vinculado à regra é recuperado como evidência. Na *Etapa 6*, a decisão é registrada com a regra, o requisito e o documento citados, além dos *digests* da especificação, da base documental e dos fatos, o que permite *replay* e auditoria. As decisões de projeto do protótipo seguem um registro curto de contexto, decisão e consequências, inspirado em registros de decisões de arquitetura [Nygard 2011].

Categorias de verificação e níveis de ablação. Para que a detecção possa ser atribuída a cada mecanismo, e não ao validador como um todo, cada problema pertence a uma categoria, e os níveis L0–L3 ativam subconjuntos cumulativos dessas categorias (Tabela 3). O nível L1 representa o versionamento convencional que qualquer equipe pode adotar; os ganhos atribuíveis à verificação SDD são, portanto, as diferenças entre L1 e L2/L3.

Tabela 3. Níveis cumulativos do validador.

Nível	Categorias ativas	Interpretação
L0	nenhuma	agente sem guarda
L1	estrutura, versão	versionamento convencional
L2	+ IDs, referências, *fingerprint*	vínculos estruturais regra–documento
L3	+ *digest*, cobertura, esquema	vínculo bidirecional e checagem de tipos

Evolução e análise de impacto. O protótipo oferece ainda operações de diferença entre versões (`diff`), matriz de rastreabilidade requisito–regra–documento (`trace`), recálculo de marcas após revisão humana (`fingerprint`) e o estudo de mutações (`mutate`). Na evolução v1→v2 do domínio de exemplo, a especificação ganha o requisito REQ-05 e a regra REG-PREMIUM-EXC (exceção para cliente *premium* que cancela até 1 hora após o despacho). A análise de impacto identifica exatamente três casos cuja decisão muda (Tabela 4).

Tabela 4. Casos impactados pela evolução v1→v2.

Caso	Decisão v1	Regra v1	Decisão v2	Regra v2
C05	NEGAR	REG-CANCEL-ENVIADO	PERMITIR	REG-PREMIUM-EXC
C06	NEGAR	REG-CANCEL-ENVIADO	PERMITIR	REG-PREMIUM-EXC
C18	NEGAR	REG-CANCEL-ENVIADO	PERMITIR	REG-PREMIUM-EXC

6. Avaliação Exploratória

Os resultados desta seção vêm do protótipo determinístico v0.1.1 (35 testes automatizados) e demonstram propriedades do código escrito. Eles não medem efeitos sobre a qualidade de LLMs e não sustentam inferência estatística: os casos são sintéticos, os rótulos e os mutantes foram escritos pelo próprio autor e há um mutante por família.

6.1. Demonstração com Cinco Condições

Dezoito casos sintéticos de reembolso e cancelamento, rotulados para v1 e v2, foram executados em cinco condições (Figura 3). As condições têm capacidades intencionalmente distintas para demonstrar o mecanismo e não constituem um ensaio causal. A

condição B3 (regras v2 com documentos v1) acerta todas as decisões, mas 16 das 18 citam evidência com versão ou *fingerprint* incompatível; as duas restantes não citam documento por usarem a decisão padrão. A mesma configuração sob a guarda SDD (P2) é bloqueada integralmente: troca evidência inválida por indisponibilidade. Com os artefatos sincronizados, a guarda não altera o resultado (B2 e P1 empatam).

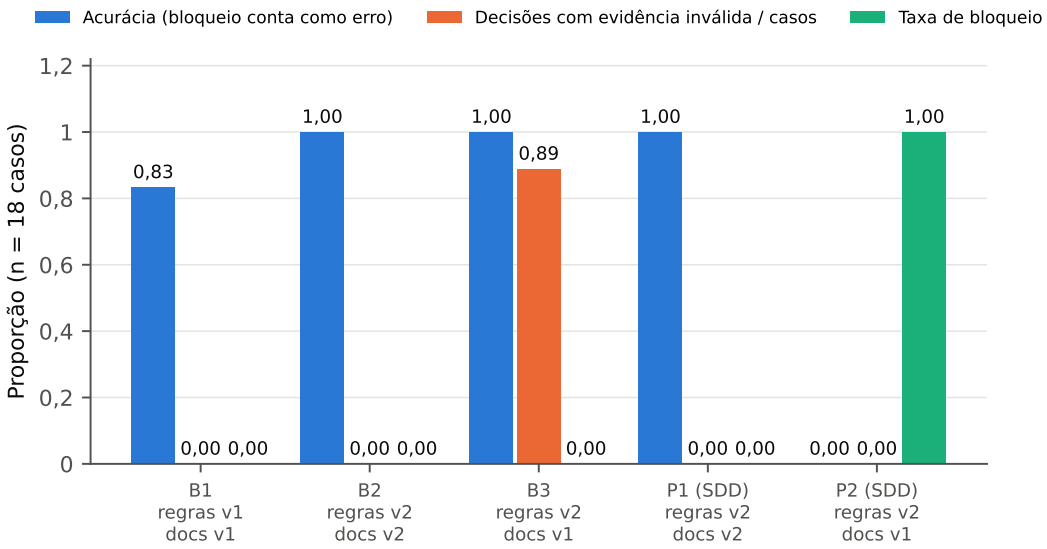


Figura 3. Acurácia, evidência inválida e bloqueio nas cinco condições da demonstração.

6.2. Estudo de Mutações

Partindo do par sincronizado (especificação v2 e documentos v2), doze operadores produzem derivas reais (M01–M12) e seis produzem mudanças benignas (N01–N06), com rótulo de verdade atribuído *a priori* (Tabela 5). Cada mutante é validado nos quatro níveis e os 18 casos são executados com guarda. A Figura 4 mostra o resultado mutante a mutante.

Tabela 5. Operadores de mutação do estudo exploratório.

ID	Deriva injetada	ID	Mudança benigna
M01	limiar alterado só na especificação	N01	reordenação de regras
M02	precedência trocada	N02	reordenação de requisitos
M03	decisão invertida	N03	reordenação de documentos
M04	regra removida	N04	metadados editoriais
M05	requisito removido	N05	correção de pontuação no texto
M06	requisito sem regra	N06	evolução completa e sincronizada
M07	justificativa contraditória		
M08	atualização parcial (um documento antigo)		
M09	texto editado sem revisão		
M10	texto invertido com marcas recalculadas		
M11	limiar alterado com marcas recalculadas		
M12	base documental v1 inteira		

A Figura 5 agrega os resultados. O versionamento convencional (L1) detecta apenas as duas derivas que mudam a versão declarada (M08 e M12). As marcas de integridade (L2) elevam a sensibilidade para 8/12, sem alarmes falsos. O nível completo

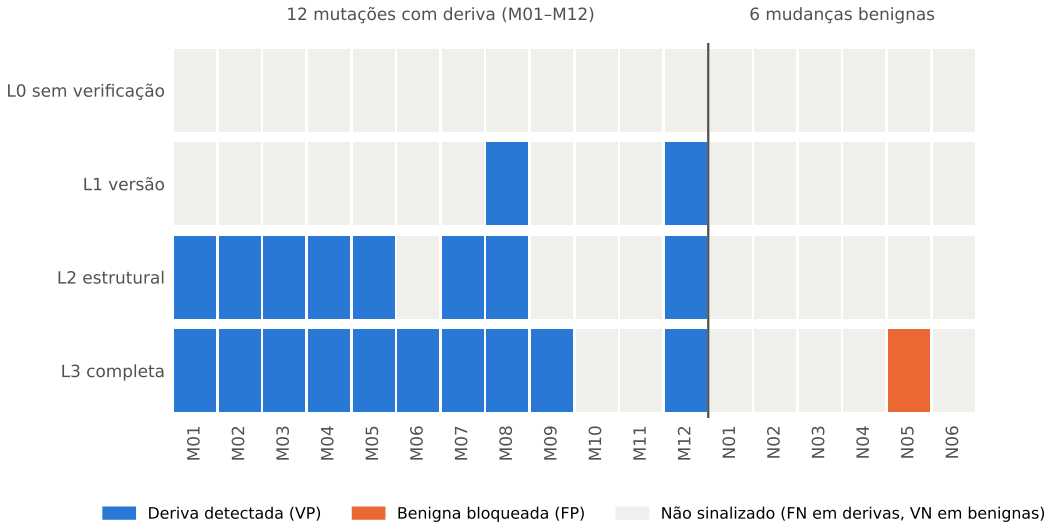


Figura 4. Detecção por mutante e por nível do validador.

(L3) chega a 10/12 ao capturar o requisito sem regra (M06) e a edição de texto sem revisão (M09), mas o *digest* de texto também bloqueia uma correção de pontuação (N05), o único alarme falso observado (precisão de 10/11).

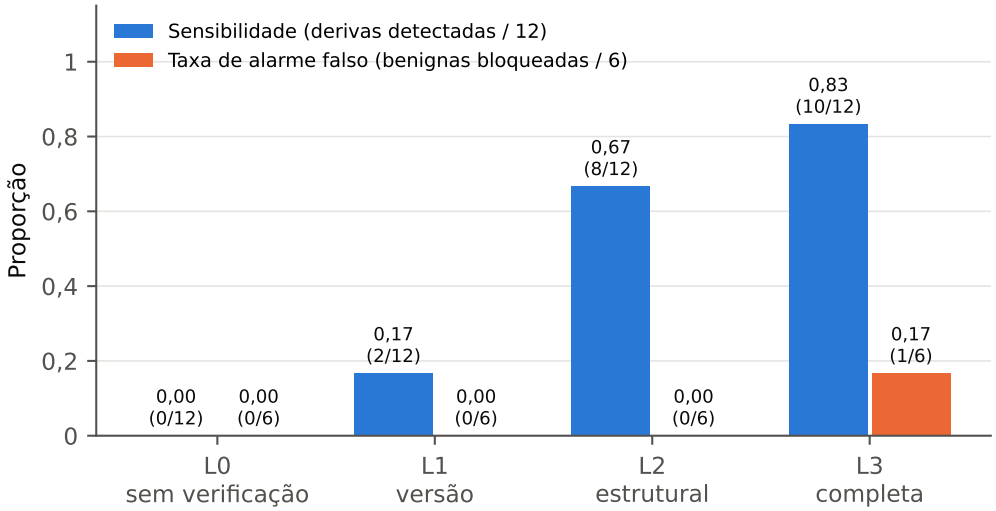


Figura 5. Sensibilidade e taxa de alarme falso por nível do validador.

A Figura 6 traduz a detecção em efeitos operacionais somados sobre os 18 mutantes. As decisões erradas emitidas caem de 9 (L0 e L1) para 1 (L2 e L3), e as decisões emitidas com evidência divergente caem de 101 para 18. O custo aparece no terceiro painel: em L3, a mudança benigna N05 bloqueia 18 decisões.

Limite semântico. M10 e M11 não são detectados em nenhum nível. Em ambos, o significado é alterado (o texto da política é invertido, ou o limiar muda mantendo o texto antigo) e as marcas de integridade são recalculadas sem revisão do conteúdo. A verificação estrutural é contornada por construção: *fingerprints* e *digests* atestam que os artefatos não mudaram desde a última aprovação, e não que texto e regra dizem a mesma

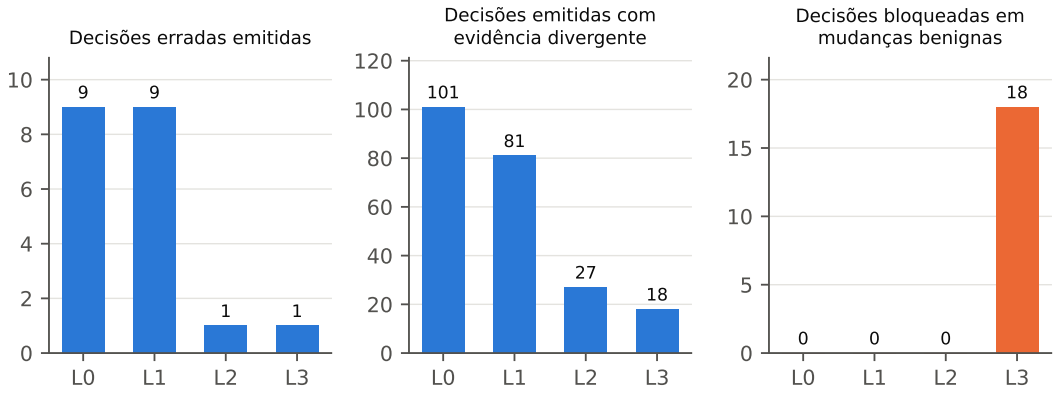


Figura 6. Efeitos operacionais por nível, somados sobre os 18 mutantes.

coisa. Esse resultado delimita o alcance da abordagem e motiva a principal extensão do estudo: avaliar se um verificador semântico (LLM como juiz, revisão humana, ou ambos) detecta essa classe de deriva e a que custo.

7. Plano de Pesquisa

Questões de pesquisa. A questão principal é: *em que medida um fluxo orientado por especificações versionadas melhora a detecção de inconsistências e a rastreabilidade de decisões de agentes híbridos ao longo da evolução de políticas de negócio?* Ela se desdobra em quatro questões:

- **QP1:** a abordagem aumenta a taxa de detecção de divergências entre especificações, regras e documentos, em comparação com *baselines* sem verificação de coevolução?
- **QP2:** a abordagem reduz decisões justificadas com evidência desatualizada durante atualizações controladas de políticas?
- **QP3:** quais custos adicionais a abordagem introduz (bloqueios, manutenção, latência e revisão humana)?
- **QP4:** em quais condições a verificação estrutural deixa passar divergências semânticas, e qual é o benefício incremental de revisão humana ou de verificação assistida por LLM?

As hipóteses são direcionais e orientam a coleta: **H1**, a verificação SDD tem maior sensibilidade do que o versionamento convencional na detecção de deriva; **H2**, ela reduz a proporção de decisões emitidas com evidência versionada incorretamente, com bloqueios contabilizados à parte; e **H3**, ela impõe custos de manutenção e de indisponibilidade mensuráveis e não desprezíveis.

Etapas. O estudo segue as diretrizes de experimentação em Engenharia de Software de [Wohlin et al. 2012] e está organizado em quatro etapas:

1. *Revisão sistemática* [Kitchenham and Charters 2007] sobre *policy-as-code*, conformidade de agentes, evolução de bases RAG, rastreabilidade em sistemas com LLM e SDD.
2. *Corpus de mudanças e operadores de mutação*, com múltiplas políticas e famílias de mudança (exceções, precedência, limiares, documentos conflitantes, inversão semântica com metadados preservados, atualização parcial e requisitos ambíguos), vários mutantes por família e rótulos atribuídos por avaliadores independentes, cegos

ao validador.

3. *Integração com LLMs e condições experimentais*, sob o mesmo modelo, corpus, limite de *tokens*, temperatura e tarefas: B1 (agente RAG sem guardas de regra); B2 (agente RAG com regras determinísticas e validação de saída); B3 (B2 com versionamento convencional); P (B3 com verificação SDD, rastreabilidade e bloqueio); e P+S (P com verificação semântica assistida por LLM). O *baseline* B3 permite atribuir os ganhos à verificação SDD, e não apenas ao versionamento; P+S responde a QP4.
4. *Análise*, com sensibilidade e precisão dos alertas, acurácia com bloqueios reportados à parte, cobertura, validade da evidência, rastreabilidade e custo (latência p50/p95, *tokens*, esforço de atualização e taxa de revisão humana). Intervalos de confiança serão estimados por *bootstrap* agrupado por cenário [Efron and Tibshirani 1993], e comparações pareadas de acerto binário usarão o teste de McNemar [McNemar 1947], com correção para comparações múltiplas. O tamanho amostral será definido por estudo piloto e análise de poder, e o protocolo será pré-registrado.

Cronograma. A Figura 7 apresenta o cronograma previsto para 24 meses. Para doutorado, as etapas 2 a 4 podem ser repetidas em mais domínios e complementadas por estudo de caso em ambiente real [Runeson and Höst 2009].

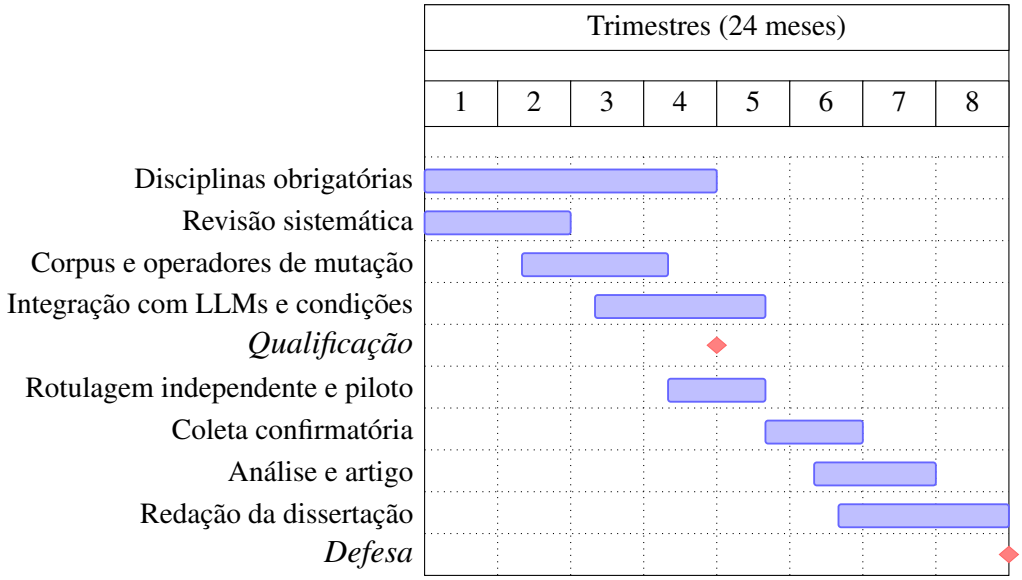


Figura 7. Cronograma previsto.

8. Ameaças à Validade

Em relação à *validade de construto*, *fingerprints* e *digests* verificam estrutura e integridade, não significado, e o oráculo exige revisão independente. Quanto à *validade interna*, um *baseline* simplificado favoreceria a proposta, por isso as condições serão pareadas em funcionalidade e recursos; além disso, mutantes escritos pelo mesmo autor do validador introduzem viés de construção, a ser mitigado com geração sistemática e rotulagem por terceiros. Quanto à *validade externa*, políticas fictícias não representam sistemas regulados reais, e o estudo buscará ao menos um cenário inspirado em documentação pública de um domínio real. Quanto à *validade de conclusão*, casos correlacionados exigem unidade de análise adequada, e o uso de poucos LLMs restringe a generalização. Por fim, do ponto de vista operacional, a política *fail-closed* aumenta bloqueios, cujo custo será medido e reportado.

9. Considerações Finais

Este projeto trata de um problema simples de enunciar e difícil de perceber na prática: quando uma política de negócio muda, a regra em código e o documento que a justifica podem deixar de dizer a mesma coisa, sem que o agente de IA apresente qualquer erro. O EvoPolicy-SDD propõe amarrar requisitos, regras e documentos numa especificação versionada e bloquear decisões quando esses artefatos divergem. O protótipo atual mostra, em pequena escala, que as marcas de integridade explicam a maior parte do ganho sobre o versionamento convencional (de 2 para 8 derivas detectadas em 12), que o *digest* de texto amplia a detecção ao custo de bloqueios por mudanças benignas, e que derivas semânticas com marcas recalculadas escapam a qualquer verificação estrutural.

Do ponto de vista científico, o trabalho pretende contribuir na interseção entre Engenharia de Software, rastreabilidade e sistemas baseados em LLMs, com um estudo controlado sobre ganhos, custos e limites da verificação de coevolução. Do ponto de vista prático, pretende oferecer um mecanismo aplicável a equipes que operam agentes sobre políticas que mudam com frequência, sem substituir a revisão humana. Como próximos passos, estão a revisão sistemática, a ampliação do corpus de mudanças e a integração com LLMs reais, incluindo a avaliação da verificação semântica como resposta ao limite identificado.

Disponibilidade dos Artefatos

O protótipo EvoPolicy-SDD v0.1.1 será disponibilizado como material suplementar e em repositório público. Esse material inclui: (i) código-fonte do pacote `evopolicy` e sua suíte de 35 testes automatizados; (ii) especificações v1 e v2, bases documentais e os 18 casos sintéticos rotulados; (iii) resultados brutos da demonstração e do estudo de mutações (`results/`), incluindo o registro auditável das decisões; e (iv) o *script* que gera as figuras deste texto a partir desses resultados.

Uso de Inteligência Artificial

Em conformidade com o Código de Conduta para Autores da SBC, declara-se que ferramentas de IA foram utilizadas como apoio. O *Claude* (Anthropic) foi utilizado como apoio à implementação do protótipo, à geração das figuras a partir dos resultados e à redação e organização deste texto, sendo o conteúdo científico, a verificação das referências e as conclusões apresentadas de responsabilidade exclusiva do autor.

Referências

Aghajani, E., Nagy, C., Vega-Márquez, O. L., Linares-Vásquez, M., Moreno, L., Bavota, G., and Lanza, M. (2019). Software documentation issues unveiled. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*, pages 1199–1210. DOI: `https://doi.org/10.1109/ICSE.2019.00122`.

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., and Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN 2024)*. arXiv:2401.05856.

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., and Yoo, S. (2015). The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5):507–525. DOI: `https://doi.org/10.1109/TSE.2014.2372785`.

Choi, J., Yoon, J., Le, L. T., Jha, S., and Pfister, T. (2026). PolicyBank: Evolving policy understanding for LLM agents. arXiv:2604.15505.

Cleland-Huang, J., Gotel, O. C. Z., Huffman Hayes, J., Mäder, P., and Zisman, A. (2014). Software traceability: Trends and future directions. In *Future of Software Engineering (FOSE 2014)*, pages 55–69. DOI: https://doi.org/10.1145/2593882.2593891.

Datla, G. V., Vurity, A., Dash, T., Ahmad, T., Adnan, M., and Rafi, S. (2025). Executable governance for AI: Translating policies into rules using LLMs. arXiv:2512.04408.

DeMillo, R. A., Lipton, R. J., and Sayward, F. G. (1978). Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41. DOI: https://doi.org/10.1109/C-M.1978.218136.

Efron, B. and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall, New York.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997.

GitHub (2025). Spec kit: toolkit for spec-driven development. https://github.com/github/spec-kit. Acesso em: 11 nov. 2026.

Gotel, O. C. Z. and Finkelstein, A. C. W. (1994). An analysis of the requirements traceability problem. In *Proceedings of the IEEE International Conference on Requirements Engineering (ICRE)*, pages 94–101. DOI: https://doi.org/10.1109/ICRE.1994.292398.

Hashmi, M., Governatori, G., Lam, H.-P., and Wynn, M. T. (2018). Are we done with business process compliance: State of the art and challenges ahead. *Knowledge and Information Systems*, 57(1):79–133. DOI: https://doi.org/10.1007/s10115-017-1142-1.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y. J., Madotto, A., and Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12). DOI: https://doi.org/10.1145/3571730.

Jia, Y. and Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678. DOI: https://doi.org/10.1109/TSE.2010.62.

Kitchenham, B. and Charters, S. (2007). Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-01, Keele University and Durham University.

Lee, H., An, G., and Yoo, S. (2025). METAMON: Finding inconsistencies between program documentation and behavior using metamorphic LLM queries. arXiv:2502.02794.

Lehman, M. M. (1980). Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076. DOI: https://doi.org/10.1109/PROC.1980.11805.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020).

Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, pages 9459–9474. Disponível em: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., and Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. DOI: https://doi.org/10.1162/tacl_a_00638.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157. DOI: https://doi.org/10.1007/BF02295996.

Meyer, B. (1992). Applying "design by contract". *Computer*, 25(10):40–51. DOI: https://doi.org/10.1109/2.161279.

National Institute of Standards and Technology (2015). Secure hash standard (SHS). Federal Information Processing Standards Publication FIPS PUB 180-4, NIST, Gaithersburg, MD. DOI: https://doi.org/10.6028/NIST.FIPS.180-4.

National Institute of Standards and Technology (2023). Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, NIST, Gaithersburg, MD. DOI: https://doi.org/10.6028/NIST.AI.100-1.

Nygard, M. (2011). Documenting architecture decisions. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions. Acesso em: 11 nov. 2026.

Open Policy Agent (s.d.). Open policy agent: Policy-based control for cloud native environments. https://www.openpolicyagent.org/docs. Acesso em: 11 nov. 2026.

Ouyang, J., Pan, T., Cheng, M., Yan, R., Luo, Y., Lin, J., and Liu, Q. (2025). HoH: A dynamic benchmark for evaluating the impact of outdated information on retrieval-augmented generation. arXiv:2503.04800.

Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., and Harman, M. (2019). Mutation testing advances: An analysis and survey. In *Advances in Computers*, volume 112, pages 275–378. Elsevier. DOI: https://doi.org/10.1016/bs.adcom.2018.03.015.

Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., and Barnes, P. (2020). Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT* '20)*, pages 33–44. DOI: https://doi.org/10.1145/3351095.3372873.

Ramesh, B. and Jarke, M. (2001). Toward reference models for requirements traceability. *IEEE Transactions on Software Engineering*, 27(1):58–93. DOI: https://doi.org/10.1109/32.895989.

Rebedea, T., Dinu, R., Sreedhar, M. N., Parisien, C., and Cohen, J. (2023). NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 431–445. Disponível em: https://aclanthology.org/2023.emnlp-demo.40.

Rinderle, S., Reichert, M., and Dadam, P. (2004). Correctness criteria for dynamic changes in workflow systems – a survey. *Data & Knowledge Engineering*, 50(1):9–34. DOI: https://doi.org/10.1016/j.datak.2004.01.002.

Runeson, P. and Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164. DOI: https://doi.org/10.1007/s10664-008-9102-8.

Schmidt, D. C. (2006). Model-driven engineering. *Computer*, 39(2):25–31. DOI: https://doi.org/10.1109/MC.2006.58.

Tan, L., Yuan, D., Krishna, G., and Zhou, Y. (2007). /* iComment: Bugs or bad comments? */. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pages 145–158. DOI: https://doi.org/10.1145/1294261.1294276.

União Europeia (2024). Regulamento (UE) 2024/1689 do parlamento europeu e do conselho, de 13 de junho de 2024, que cria regras harmonizadas em matéria de inteligência artificial (regulamento da inteligência artificial). Jornal Oficial da União Europeia, Série L, 12 jul. 2024. Disponível em: https://eur-lex.europa.eu/eli/reg/2024/1689/oj.

Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., and Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6). DOI: https://doi.org/10.1007/s11704-024-40231-1.

Wen, F., Nagy, C., Bavota, G., and Lanza, M. (2019). A large-scale empirical study on code-comment inconsistencies. In *Proceedings of the 27th IEEE/ACM International Conference on Program Comprehension (ICPC)*, pages 53–64. Disponível em: https://www.inf.usi.ch/faculty/lanza/PUBS/P/Wen2019a.pdf.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., and Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg. DOI: https://doi.org/10.1007/978-3-642-29044-2.

Xu, X., Wahab, Z., Holmes, R., and Lemieux, C. (2025). DocPrism: Multi-lingual detection of incorrectness inconsistencies between code and documentation. arXiv:2511.00215.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*. arXiv:2210.03629.

Zaidman, A., Van Rompaey, B., van Deursen, A., and Demeyer, S. (2011). Studying the co-evolution of production and test code in open source and industrial developer test processes through repository mining. *Empirical Software Engineering*, 16(3):325–364. DOI: https://doi.org/10.1007/s10664-010-9143-7.

Zwerdling, N., Boaz, D., Rabinovich, E., Uziel, G., and Anaby-Tavor, A. (2025). Towards enforcing company policy adherence in agentic workflows. arXiv:2507.16459.